```
const fs = require('node:fs/promises');
const http = require('node:http');
const os = require('node:os');
const path = require('node:path');

const DEFAULT_PORT = 3037;
const DEFAULT_PRINTER_NAME = 'POS-80-Series';
const MAX_BODY_SIZE = 4096;
const COUNTER_LOCK_TIMEOUT_MS = 30000;
const COUNTER_LOCK_RETRY_MS = 100;
const STUDENTS_PATH = path.join(__dirname,
'alumnos.json');
const CONFIG_PATH = path.join(__dirname,
'cajero-config.json');
const COUNTER_PATH = path.join(__dirname,
'contador-tickets.json');
const LEGACY_COUNTER_PATH = path.join(
  process.env.LOCALAPPDATA || os.homedir(),
  'eBANK',
  'contador-tickets.json'
);
const CONFIG_LIMITS = {
  atmWidth: [640, 1920],
  atmHeight: [480, 1200],
  videoWidth: [300, 1600],
  videoHeight: [150, 900],
  sideColumnWidth: [15, 45],
  buttonGap: [0, 300],
  buttonTopMargin: [0, 300],
  buttonFontSize: [12, 40],
  buttonHeight: [40, 100],
  contentTextFontSize: [12, 40],
  amountFontSize: [16, 60],
  inactivitySeconds: [5, 600],
  processCarouselIntervalSeconds: [1, 60],
  participantsCarouselIntervalSeconds: [1,
60],
  carouselPauseSeconds: [0, 120],
  carouselInactivitySeconds: [5, 600],
  processCarouselWidth: [30, 100],
  processCarouselHeight: [150, 700],
  participantsCarouselWidth: [30, 100],
  participantsCarouselHeight: [150, 700]
};
const CONFIG_KEYS =
Object.keys(CONFIG_LIMITS);
const ALLOWED_ORIGINS =
(process.env.EBANK_ALLOWED_ORIGINS ||
'http://localhost,http://127.0.0.1')
  .split(',')
  .map((origin) => origin.trim())
```

```
  .filter(Boolean);
function validateTicketData(data) {
  if (!data || typeof data !== 'object' ||
Array.isArray(data)) {
    const error = new Error('El pedido de
impresión no tiene un formato válido.');
    error.statusCode = 400;
    throw error;
  }
  if (Object.keys(data).some((key) => key !==
'dni')) {
    const error = new Error('El pedido solo
debe incluir el DNI; el nombre se consulta en
el registro local.');
    error.statusCode = 400;
    throw error;
  }

  if (typeof data.dni !== 'string' ||
!/^\d{7,8}$/.test(data.dni)) {
    const error = new Error('El DNI debe
tener 7 u 8 números.');
    error.statusCode = 400;
    throw error;
  }

  return { dni: data.dni };
}

async function findRegisteredStudentName(dni,
studentsPath = STUDENTS_PATH) {
  const students = JSON.parse(await
fs.readFile(studentsPath, 'utf8'));
  if (!Array.isArray(students)) {
    throw new Error('El archivo de alumnos
debe contener una lista.');
  }

  const student = students.find((entry) =>
entry && String(entry.dni) === dni);
  if (!student) return '';

  const name = `${student.nombre || ''}
${student.apellido || ''}`.trim();
  if (!name || name.length > 80 ||
!/^[\p{L}\p{M} .'-]+$/u.test(name) ||
      [...name].some((character) =>
character.codePointAt(0) > 255)) {
    throw new Error(`El nombre del alumno con
DNI ${dni} no se puede imprimir en la
impresora configurada.`);
```

```
  }
  return name;
}

function buildWelcomeTicket({ ticketNumber,
dni, name, date = new Date(),
logoRasterBuffer = null }) {
  const ESC = '\x1b';
  const GS = '\x1d';
  const center = ESC + 'a' + '\x01';
  const left = ESC + 'a' + '\x00';
  const boldOn = ESC + 'E' + '\x01';
  const boldOff = ESC + 'E' + '\x00';

  const largeText = GS + '!' + '\x11';   //
Doble ancho y doble alto (para el título
principal)
  const mediumText = GS + '!' + '\x01';  //
Doble altura (tamaño intermedio ideal para
Nombre y DNI)
  const normalText = GS + '!' + '\x00';  //
Texto normal

  const codePage1252 = ESC + 't' + '\x10';
  const cutPaper = GS + 'V' + '\x41' +
'\x03';

  const timestamp = new
Intl.DateTimeFormat('es-AR', {
    dateStyle: 'short',
    timeStyle: 'short',
    timeZone:
'America/Argentina/Buenos_Aires'
  }).format(date);

  let ticket = ESC + '@' + codePage1252;

  // 1. Logo opcional en formato raster (si
se pasa el buffer de la imagen procesada)
  if (logoRasterBuffer) {
    ticket += center;
    ticket += logoRasterBuffer;
    ticket += '\n';
  }

  // 2. Título de bienvenida grande y
centrado
  ticket += center + largeText + boldOn;
  ticket += 'BIENVENIDO/A A LA\nEXPO TECNO
CONTABLE\nISD 2026\n\n';
```

```
  // 3. Nombre y DNI más grandes (tamaño
intermedio), en líneas separadas
  ticket += center + mediumText + boldOn;
  if (name) {
    ticket += `NOMBRE:\n${name}\n\n`;
  }
  ticket += `DNI:\n${dni}\n\n`;

  // 4. Fecha, hora y correlativo en tamaño
normal y alineados a la izquierda
  ticket += normalText + boldOff + left;
  ticket += `FECHA Y HORA: ${timestamp}\n`;
  ticket += `NRO. CORRELATIVO:
${String(ticketNumber).padStart(6, '0')}\n`;

  // 5. Pie de página
  ticket += '\n' + center + 'Gracias por usar
nuestro\ncajero eBANK - ISD\n\n\n\n';
  ticket += cutPaper;

  // Si manejas texto y un buffer raster de
imagen juntos, es mejor combinarlos como
Buffer de Node.js:
  if (logoRasterBuffer) {
    // Si usas buffer raster para la imagen,
se suele concatenar a nivel de Buffers
directamente.
    // Pero si `logoRasterBuffer` es un
string ESC/POS válido, funcionará con la
concatenación de arriba.
  }

  return Buffer.from(ticket, 'latin1');
}

async function acquireCounterLock(lockPath) {
  const deadline = Date.now() +
COUNTER_LOCK_TIMEOUT_MS;
  while (Date.now() < deadline) {
    try {
      const handle = await fs.open(lockPath,
'wx');
      await
handle.writeFile(String(process.pid),
'utf8');
      return handle;
    } catch (error) {
      if (error.code !== 'EEXIST') throw
error;

      let lockContents = '';
```

```
      try {
        lockContents = await
fs.readFile(lockPath, 'utf8');
      } catch (readError) {
        if (readError.code === 'ENOENT')
continue;
        throw readError;
      }

      const ownerPid = Number(lockContents);
      if (!lockContents.trim()) {
        const lockStats = await
fs.stat(lockPath);
        if (Date.now() - lockStats.mtimeMs >
1000) {
          try {
            await fs.unlink(lockPath);
          } catch (unlinkError) {
            if (unlinkError.code !==
'ENOENT') throw unlinkError;
          }
          continue;
        }
      }
      if (Number.isSafeInteger(ownerPid) &&
ownerPid > 0) {
        let ownerIsAlive = true;
        try {
          process.kill(ownerPid, 0);
        } catch (processError) {
          if (processError.code === 'ESRCH')
ownerIsAlive = false;
          else if (processError.code !==
'EPERM') throw processError;
        }

        if (!ownerIsAlive) {
          try {
            await fs.unlink(lockPath);
          } catch (unlinkError) {
            if (unlinkError.code !==
'ENOENT') throw unlinkError;
          }
          continue;
        }
      }

      await new Promise((resolve) =>
setTimeout(resolve, COUNTER_LOCK_RETRY_MS));
    }
  }
```

```
  throw new Error('No se pudo reservar un
número correlativo: otro servicio mantiene
ocupado el contador.');
}

async function
reserveTicketNumber(counterPath, dni) {
  await fs.mkdir(path.dirname(counterPath), {
recursive: true });
  const lockPath = `${counterPath}.lock`;
  const lockHandle = await
acquireCounterLock(lockPath);

  try {
    const { lastNumber: currentNumber,
usedDnis } = await
readCounterState(counterPath);

    if
(Object.prototype.hasOwnProperty.call(usedDni
s, dni)) {
      return { alreadyUsed: true,
ticketNumber: usedDnis[dni] };
    }

    if (currentNumber >=
Number.MAX_SAFE_INTEGER) {
      throw new Error('El contador de tickets
llegó al límite permitido.');
    }

    const number = currentNumber + 1;
    const updatedUsedDnis = { ...usedDnis,
[dni]: number };
    await writeCounterState(counterPath, {
lastNumber: number, usedDnis: updatedUsedDnis
});
    return { alreadyUsed: false,
ticketNumber: number };
  } finally {
    try {
      await lockHandle.close();
    } finally {
      await fs.unlink(lockPath);
    }
  }
}

async function readCounterState(counterPath)
{
```

```
  let stored;
  try {
    stored = JSON.parse(await
fs.readFile(counterPath, 'utf8'));
  } catch (error) {
    if (error.code === 'ENOENT') return {
lastNumber: 0, usedDnis: {} };
    throw error;
  }

  if (!stored ||
!Number.isSafeInteger(stored.lastNumber) ||
stored.lastNumber < 0) {
    throw new Error('El contador persistente
de tickets no contiene un número válido. No
se modificó para evitar repetir números.');
  }
  if (
    stored.usedDnis !== undefined &&
    (!stored.usedDnis || typeof
stored.usedDnis !== 'object' ||
Array.isArray(stored.usedDnis))
  ) {
    throw new Error('El registro persistente
de DNI no es válido. No se modificó para
evitar imprimir duplicados.');
  }

  const usedDnis = stored.usedDnis || {};
  for (const [registeredDni, ticketNumber] of
Object.entries(usedDnis)) {
    if (!/^\d{7,8}$/.test(registeredDni) ||
!Number.isSafeInteger(ticketNumber) ||
        ticketNumber < 1 || ticketNumber >
stored.lastNumber) {
      throw new Error('El registro
persistente de DNI o tickets está corrupto.
No se modificó para evitar duplicados.');
    }
  }
  return { lastNumber: stored.lastNumber,
usedDnis };
}

async function writeCounterState(counterPath,
state) {
  const temporaryPath =
`${counterPath}.${process.pid}.${Date.now()}.
tmp`;
  try {
    await fs.writeFile(temporaryPath,
```

```
JSON.stringify(state), { encoding: 'utf8',
flag: 'wx' });
    await fs.rename(temporaryPath,
counterPath);
  } catch (error) {
    try {
      await fs.unlink(temporaryPath);
    } catch (cleanupError) {
      if (cleanupError.code !== 'ENOENT') {
        console.error('No se pudo limpiar el
archivo temporal del contador.',
cleanupError);
      }
    }
    throw error;
  }
}

async function listUsedDnis(counterPath) {
  const { usedDnis } = await
readCounterState(counterPath);
  return Object.entries(usedDnis)
    .map(([dni, ticketNumber]) => ({ dni,
ticketNumber }))
    .sort((first, second) =>
first.ticketNumber - second.ticketNumber);
}

async function
findUsedTicketNumber(counterPath, dni) {
  const { usedDnis } = await
readCounterState(counterPath);
  return
Object.prototype.hasOwnProperty.call(usedDnis
, dni) ? usedDnis[dni] : null;
}

async function writeToPrinter(printerName,
ticket) {
  const printerShare =
`\\\\localhost\\${printerName}`;
  await fs.writeFile(printerShare, ticket);
}

function sendJson(response, statusCode,
payload) {
  response.writeHead(statusCode, {
    'Content-Type': 'application/json;
charset=utf-8',
    'Cache-Control': 'no-store'
  });
```

```
  response.end(JSON.stringify(payload));
}

async function readJsonBody(request) {
  const chunks = [];
  let size = 0;

  for await (const chunk of request) {
    size += chunk.length;
    if (size > MAX_BODY_SIZE) {
      const error = new Error('El pedido de
impresión supera el tamaño permitido.');
      error.statusCode = 413;
      throw error;
    }
    chunks.push(chunk);
  }

  try {
    return
JSON.parse(Buffer.concat(chunks).toString('ut
f8'));
  } catch {
    const error = new Error('El pedido de
impresión no contiene JSON válido.');
    error.statusCode = 400;
    throw error;
  }
}

function validateCajeroConfig(data) {
  if (!data || typeof data !== 'object' ||
Array.isArray(data)) {
    const error = new Error('La configuración
debe ser un objeto JSON.');
    error.statusCode = 400;
    throw error;
  }

  const config = {};
  for (const key of CONFIG_KEYS) {
    const value = data[key];
    const [minimum, maximum] =
CONFIG_LIMITS[key];
    if (typeof value !== 'number' ||
!Number.isFinite(value) || value < minimum ||
value > maximum) {
      const error = new Error(`El valor de
configuración "${key}" debe ser un número
entre ${minimum} y ${maximum}.`);
      error.statusCode = 400;
```

```
      throw error;
    }
    config[key] = value;
  }
  if (Object.keys(data).some((key) =>
!CONFIG_KEYS.includes(key))) {
    const error = new Error('La configuración
contiene opciones desconocidas.');
    error.statusCode = 400;
    throw error;
  }
  return config;
}

async function readCajeroConfig(configPath) {
  try {
    return JSON.parse(await
fs.readFile(configPath, 'utf8'));
  } catch (error) {
    if (error.code === 'ENOENT') return null;
    throw error;
  }
}

async function writeCajeroConfig(configPath,
config) {
  const temporaryPath =
`${configPath}.${process.pid}.${Date.now()}.t
mp`;
  try {
    await fs.writeFile(temporaryPath,
JSON.stringify(config, null, 2) + '\n', {
      encoding: 'utf8',
      flag: 'wx'
    });
    await fs.rename(temporaryPath,
configPath);
  } catch (error) {
    try {
      await fs.unlink(temporaryPath);
    } catch (cleanupError) {
      if (cleanupError.code !== 'ENOENT') {
        console.error('No se pudo limpiar el
archivo temporal de configuración.',
cleanupError);
      }
    }
    throw error;
  }
}
```

```
async function migrateCounterFile(sourcePath,
destinationPath) {
  if (path.resolve(sourcePath) ===
path.resolve(destinationPath)) return false;

  try {
    await fs.access(destinationPath);
    return false;
  } catch (error) {
    if (error.code !== 'ENOENT') throw error;
  }

  let sourceContents;
  try {
    sourceContents = await
fs.readFile(sourcePath);
  } catch (error) {
    if (error.code === 'ENOENT') return
false;
    throw error;
  }

  await
fs.mkdir(path.dirname(destinationPath), {
recursive: true });
  const temporaryPath =
`${destinationPath}.${process.pid}.${Date.now
()}.migration.tmp`;
  try {
    await fs.writeFile(temporaryPath,
sourceContents, { flag: 'wx' });
    await fs.link(temporaryPath,
destinationPath);
  } catch (error) {
    if (error.code === 'EEXIST') return
false;
    throw error;
  } finally {
    try {
      await fs.unlink(temporaryPath);
    } catch (error) {
      if (error.code !== 'ENOENT') {
        console.error('No se pudo limpiar el
archivo temporal de migración del contador.',
error);
      }
    }
  }

  const copiedContents = await
fs.readFile(destinationPath);
```

```
  if (!copiedContents.equals(sourceContents))
{
    throw new Error('La copia del registro de
tickets no coincide con el archivo
original.');
  }

  try {
    await fs.unlink(sourcePath);
  } catch (error) {
    console.warn(`El registro ya fue copiado,
pero no se pudo quitar el archivo anterior
${sourcePath}:`, error);
  }
  return true;
}

function createPrintServer(options = {}) {
  const printerName = options.printerName ||
process.env.NOMBRE_IMPRESORA ||
DEFAULT_PRINTER_NAME;
  const counterPath = options.counterPath ||
COUNTER_PATH;
  const configPath = options.configPath ||
CONFIG_PATH;
  const studentsPath = options.studentsPath
|| STUDENTS_PATH;
  const allowedOrigins =
options.allowedOrigins || ALLOWED_ORIGINS;
  const print = options.print || ((ticket) =>
writeToPrinter(printerName, ticket));
  const logError = options.logError ||
console.error;
  let printQueue = Promise.resolve();

  if (!/^[\w .-]{1,80}$/.test(printerName)) {
    throw new Error('El nombre compartido de
la impresora contiene caracteres no
permitidos.');
  }

  const server = http.createServer(async
(request, response) => {
    const address = server.address();
    if (!address) {
      sendJson(response, 503, { error: 'El
servicio de impresión no está listo.' });
      return;
    }
    const allowedHosts =
[`127.0.0.1:${address.port}`,
```

```
`localhost:${address.port}`];
    if
(!allowedHosts.includes(request.headers.host)
) {
      sendJson(response, 403, { error: 'El
servicio solo acepta conexiones locales.' });
      return;
    }

    const origin = request.headers.origin;
    if (!origin ||
!allowedOrigins.includes(origin)) {
      sendJson(response, 403, { error:
'Origen no autorizado. Revisá
EBANK_ALLOWED_ORIGINS.' });
      return;
    }


response.setHeader('Access-Control-Allow-Orig
in', origin);
    response.setHeader('Vary', 'Origin');

response.setHeader('Access-Control-Allow-Meth
ods', 'GET, POST, OPTIONS');

response.setHeader('Access-Control-Allow-Head
ers', 'Content-Type');

    if (request.method === 'OPTIONS' &&
        ['/api/tickets', '/api/tickets/used',
'/api/tickets/reprint',
'/api/config'].includes(request.url)) {
      response.writeHead(204);
      response.end();
      return;
    }

    if (request.method === 'GET' &&
request.url === '/api/config') {
      try {
        sendJson(response, 200, { config:
await readCajeroConfig(configPath) });
      } catch (error) {
        logError('No se pudo leer la
configuración compartida del cajero:',
error);
        sendJson(response, 500, { error: `No
se pudo leer la configuración:
${error.message}` });
      }
```

```
      return;
    }

    if (request.method === 'POST' &&
request.url === '/api/config') {
      if
(!request.headers['content-type']?.toLowerCas
e().startsWith('application/json')) {
        sendJson(response, 415, { error: 'El
servicio requiere un pedido JSON.' });
        return;
      }
      try {
        const config =
validateCajeroConfig(await
readJsonBody(request));
        await writeCajeroConfig(configPath,
config);
        sendJson(response, 200, { ok: true,
config });
      } catch (error) {
        logError('No se pudo guardar la
configuración compartida del cajero:',
error);
        sendJson(response, error.statusCode
|| 500, {
          error: error.statusCode ?
error.message : `No se pudo guardar la
configuración: ${error.message}`
        });
      }
      return;
    }

    if (request.method === 'GET' &&
request.url === '/api/tickets/used') {
      try {
        sendJson(response, 200, { usedDnis:
await listUsedDnis(counterPath) });
      } catch (error) {
        logError('No se pudo leer el registro
de tickets utilizados:', error);
        sendJson(response, 500, { error: `No
se pudo leer el registro: ${error.message}`
});
      }
      return;
    }

    if (request.method === 'POST' &&
request.url === '/api/tickets/reprint') {
```

```
      if
(!request.headers['content-type']?.toLowerCas
e().startsWith('application/json')) {
        sendJson(response, 415, { error: 'El
servicio requiere un pedido JSON.' });
        return;
      }
      try {
        const { dni } =
validateTicketData(await
readJsonBody(request));
        const ticketNumber = await
findUsedTicketNumber(counterPath, dni);
        if (ticketNumber === null) {
          sendJson(response, 404, { error:
'Este DNI no tiene un ticket previo para
reimprimir.' });
          return;
        }

        const queuedPrint =
printQueue.then(async () => {
          const name = await
findRegisteredStudentName(dni, studentsPath);
          await print(buildWelcomeTicket({
dni, name, ticketNumber }));
          return ticketNumber;
        });
        printQueue = queuedPrint.catch(() =>
{});
        sendJson(response, 200, { ok: true,
reprinted: true, ticketNumber: await
queuedPrint });
      } catch (error) {
        logError('No se pudo reimprimir el
ticket ya asignado:', error);
        sendJson(response, error.statusCode
|| 500, {
          error: error.statusCode ?
error.message : `No se pudo reimprimir:
${error.message}`
        });
      }
      return;
    }

    if (request.method !== 'POST' ||
request.url !== '/api/tickets') {
      sendJson(response, 404, { error: 'Ruta
de impresión no encontrada.' });
      return;
```

```
    }

    if
(!request.headers['content-type']?.toLowerCas
e().startsWith('application/json')) {
      sendJson(response, 415, { error: 'El
servicio requiere un pedido JSON.' });
      return;
    }

    try {
      const ticketData =
validateTicketData(await
readJsonBody(request));
      const queuedPrint =
printQueue.then(async () => {
        const name = await
findRegisteredStudentName(ticketData.dni,
studentsPath);
        const reservation = await
reserveTicketNumber(counterPath,
ticketData.dni);
        if (reservation.alreadyUsed) return
reservation;

        const ticketNumber =
reservation.ticketNumber;
        const ticket = buildWelcomeTicket({
...ticketData, name, ticketNumber });
        await print(ticket);
        return { alreadyUsed: false,
ticketNumber };
      });
      printQueue = queuedPrint.catch(() =>
{});

      const result = await queuedPrint;
      if (result.alreadyUsed) {
        sendJson(response, 409, {
          alreadyUsed: true,
          ticketNumber: result.ticketNumber,
          error: 'Este DNI ya fue utilizado
para imprimir un ticket.'
        });
        return;
      }
      sendJson(response, 200, { ok: true,
ticketNumber: result.ticketNumber });
    } catch (error) {
      logError('Error al generar o imprimir
el ticket de bienvenida:', error);
```

```
      sendJson(response, error.statusCode ||
500, {
        error: error.statusCode ?
error.message : `No se pudo imprimir:
${error.message}`
      });
    }
  });

  return server;
}

if (require.main === module) {
  const port = Number(process.env.EBANK_PORT
|| DEFAULT_PORT);
  if (!Number.isInteger(port) || port < 1 ||
port > 65535) {
    console.error('EBANK_PORT debe ser un
puerto entre 1 y 65535.');
    process.exitCode = 1;
  } else {
    migrateCounterFile(LEGACY_COUNTER_PATH,
COUNTER_PATH).then((migrated) => {
      if (migrated) console.log('Registro de
DNI y correlativos migrado a la carpeta del
cajero.');
      const server = createPrintServer();
      server.listen(port, '127.0.0.1', () =>
{
        console.log(`Servicio de impresión
eBANK activo en http://127.0.0.1:${port}`);
        console.log(`Impresora compartida
configurada: ${process.env.NOMBRE_IMPRESORA
|| DEFAULT_PRINTER_NAME}`);
        console.log(`Orígenes web
autorizados: ${ALLOWED_ORIGINS.join(', ')}`);
        console.log(`Registro de DNI y
correlativos: ${COUNTER_PATH}`);
      });
      server.on('error', (error) => {
        console.error('No se pudo iniciar el
servicio de impresión eBANK:', error);
        process.exitCode = 1;
      });
    }).catch((error) => {
      console.error('No se pudo migrar el
registro de DNI y correlativos:', error);
      process.exitCode = 1;
    });
  }
}
```

```
module.exports = {
  buildWelcomeTicket,
  createPrintServer,
  findUsedTicketNumber,
  listUsedDnis,
  migrateCounterFile,
  reserveTicketNumber,
  validateCajeroConfig,
  validateTicketData
};
```